New Mexico State University

Department of Computer Science

CSCI 4140 / 5140 — Database Management Systems

Group Research Project

Phase 1 Report

Neo4j: A Native Property Graph Database

Data Model, Query Language, and Readiness for Experimental Comparison with MySQL

Team 6

Rigesh Krishnaraj

Suresh Hemal Warnasooriya Warnakulasooriya Mudiyanselage

Scott C. Robertson

Instructor: Dr. Parth Nagarkar

Semester: Fall 2026

Submitted: September 28th, 2026

Submitted in partial fulfilment of the requirements for CSCI 4140 / 5140

Contents

1 Introduction **1**

2 Data Model Fit **1**
2.1 Background: Data Models in the DBMS Landscape; A Brief Introduction 1
2.2 Mental Model: MySQL → Neo4j (using the el_paso_airport schema) 2
2.3 Data Assumptions 3
2.4 Natural Applications Supporting 3
2.5 Struggles 4

3 Query Language and Interface **4**
3.1 Query and Programming Languages 4
3.2 SQL Support 4
3.3 Key Differences 4
3.4 Worked Examples: el_paso Airline Database (MySQL) vs Neo4j (Cypher) 5

4 Use Cases and Practical Application **9**
4.1 Business Use Cases 9
4.2 Industry Use Cases 10
4.3 Healthcare and Life Science 10
4.4 Real Use Case: WestJet 10
4.5 Companies Using Neo4j 11
4.6 How and Why Does Neo4j Work Better for Their Use Cases Compared to Traditional Relational Databases? 11

5 Competitors and Market Analysis **12**
5.1 ArcadeDB 12
5.2 Memgraph 12
5.3 How Does Neo4j Differ? 12
5.4 What Makes Neo4j Unique? 12

6 Drawbacks and Limitations **13**
6.1 Memory Bound on a Single Machine 13
6.2 Hard-To-Predict Query Performance 13

7 Phase 2 Readiness and Reflection **13**

8 Group Members and Contributions **14**

References **15**

List of Tables

1 Non-relational data models grouped into categories, after Stonebraker and Pavlo [1]. 1
2 Mapping of MySQL concepts to Neo4j concepts, using the el_paso_airport schema. 2
3 The el_paso_airport schema and dataset size, verified with DESCRIBE (MySQL 8.4). 5
4 Basic CRUD operations in MySQL and the equivalent Cypher statements. 6

List of Listings

1 Property graph model for the el_paso_airport data 5
2 Selecting staff by position in SQL 6
3 Selecting staff by position in Cypher 6
4 Trips booked by one passenger in July 2019, in SQL 7
5 The same query in Cypher, filtering in a separate WHERE clause 7
6 Filtering with a property map inside the pattern 7
7 Filtering with an inline WHERE inside the relationship brackets 7
8 Pilots who flew exactly once in July 2019, in SQL 8
9 The same query in Cypher, using WITH in place of GROUP BY and HAVING . 8
10 Passengers to El Paso with more than 2000 reward points, in SQL 8
11 The same query in Cypher, two nested sub-queries collapsed into one pattern . . 8
12 Services on aircraft flown by one pilot, in SQL: four levels of nested sub-queries . 8
13 The same question in Cypher, expressed as a single path 8
14 High-capacity trips from towns beginning with S, in SQL, using NATURAL JOIN 9
15 The same query in Cypher, naming the relationship explicitly 9
16 A variable-length path in Cypher, which SQL expresses with a recursive CTE . . 9

1 Introduction

The database in question that we chose for the Database Management Systems project is Neo4j which was created by Emil Eifrem, Johan Svensson, and Peter Neubauer in Sweden [2]. The company in which it was created was originally called Neo Technology and was eventually renamed to Neo4j, Inc. on July 27th, 2017 [3]. The name can be split into two parts. The first part, "Neo", means new and the second part, "4j" is short for "for java". Making the full name "New For Java". The project was also originally called "Project Neo".

Neo4j was created in 2007 along with the company in Sweden. Neo4j, was released as open source [2, 4]. Later on, in February 2010, the company officially released version 1.0 to the general public. In 2011, the company moved and set up shop in current Silicon Valley, California, and they kept their engineering process in Sweden [2, 4].

The idea of Neo4j started off in the year 2000, while on a flight to Mumbai, where Eifrem sketched the first property graph of the now Neo4j on a napkin. The three founders at the time were building an enterprise content management system (ECM) and kept going against the challenge of using an RDBMS for querying connected data. Before the idea of Neo4j, Eifrem was originally at Windh Technologies, which is a media asset company [5].

The core idea of the company is to store relationships as first class entities instead of re-building them with joins [4]. The company has its roots in Malmö, Sweden where development still occurs. Today, their headquarters is in San Mateo, California [6].

2 Data Model Fit

2.1 Background: Data Models in the DBMS Landscape; A Brief Introduction

Table 1 groups non-relational data into eight different categories [1], together with the relational model for comparison.

Table 1: Non-relational data models grouped into categories, after Stonebraker and Pavlo [1].

Model	**Shape of Data**	**Examples**	**Paper's View**
Relational (RM)	Tables of rows and columns, queried with SQL	MySQL, PostgreSQL, Oracle	Dominant; most other models converge back towards it
MapReduce	No fixed model; user-written Map and Reduce functions over files	Google MR, Hadoop	"Hadoop died about a decade ago"; replaced by Spark, Flink
Key/value	(key, value); value is an opaque blob; get/set/delete	Memcached, Redis, DynamoDB, RocksDB	Good for caching and sessions; "a bad idea" when records need multiple fields
Document	Nested field/value hierarchies (JSON)	MongoDB	Nearly all added SQL and ACID; "will soon be effectively identical" to RDBMSs
Column-family (wide-column)	Records keyed by ID with optional attributes; one level of nesting	BigTable, Cassandra, HBase	Niche; lacks joins and secondary indexes
Text search	Inverted indexes over tokenized words	Elasticsearch, Solr (Lucene)	Limited transactions; RDBMSs now offer full-text search
Vector	One-dimensional embeddings for similarity search	Pinecone, Milvus, Weaviate	RDBMSs are quickly adding vector indexes

Model	Shape of Data	Examples	Paper's View
Graph	Nodes and edges	Neo4j, TigerGraph, JanusGraph	"We do not expect specialized graph DBMSs to be a large market"

There are two ways of representing graphs [1]:

- RDF (triplestores): a directed graph with labeled edges only (subject → predicate → object).
- Property graph: a directed multi-graph with key/value properties on nodes and edges. It is more common, and the paper calls it "a superset of RDF".

In this case, Neo4j uses the property graph model.

2.2 Mental Model: MySQL → Neo4j (using the el_paso_airport schema)

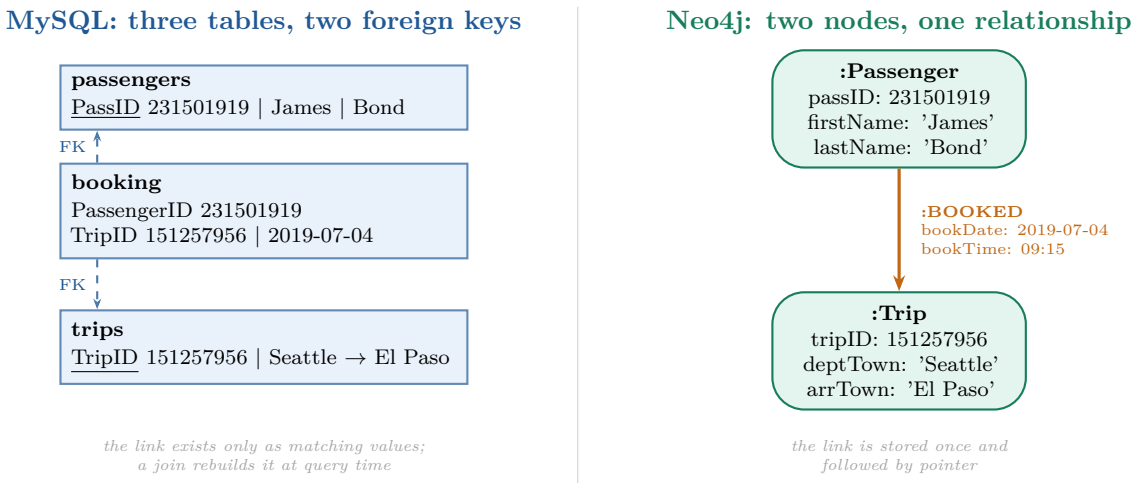


Figure 1: One booking, stored two ways. In MySQL the connection between a passenger and a trip lives in the `booking` table as two foreign-key values that a join must match at query time. In Neo4j the same connection is a stored `:BOOKED` relationship that carries its own properties and is followed directly.

Figure 1 shows the same booking stored both ways, and Table 2 summarizes how each MySQL concept maps onto Neo4j. In one line: labels are like table names, except a node can have several; a relationship is a foreign key or a pivot-table row promoted to a real object that can carry its own columns.

Table 2: Mapping of MySQL concepts to Neo4j concepts, using the el_paso_airport schema.

MySQL	Neo4j	Note
Table name (pilots)	Label (:Pilot)	A node may have zero or more labels [7].
Row	Node	
Column	Property	Property lives on nodes and relationships [7].
Foreign key column (trips.AirCraftID)	Relationship (:Trip)-[:USES]->(:Aircraft)	A single FK becomes a relationship too, not only pivot tables.

MySQL	**Neo4j**	**Note**
Pivot table without extra columns (flies)	Relationship (:Pilot)-[:FLIES]->(:Trip)	The table disappears entirely.
Pivot table with extra columns (works(StaffID, DepartmentID, Year))	(:Staff)-[:WORKS_IN {year: 2019}]->(:Department)	Extra columns become relationship properties.
booking(PassengerID, TripID, BookDate, BookTime)	(:Passenger)-[:BOOKED {bookDate, bookTime}]->(:Trip)	Composite PK = the relationship itself.
Subtype tables / UNION	Multiple labels (:Person:Pilot, :Person:Staff)	MATCH (person:Person) then matches both.

As the last row of Table 2 shows, it is worthy to note in this report that a node can carry many labels, but a relationship can have exactly one type plus a direction, a start node, and an end node [7]. Concrete payoff in this schema: pilots and staff repeat in the same nine columns (FirstName, LastName, Birthdate, Address, Email, PhoneNumber, Gender, Salary, HiredDate) however a shared :Person label will remove that duplication and simplifies the case of staff and pilots sharing a first name, which in SQL needs a join across the two tables. When it comes to open modeling question two, if a booking needs to store its own information, (seat number, price paid, check in time, flight status, etc.) should it become its own object, a booking node between them? This is the same as the question in traditional databases of when a connecting table should become an entity of its own.

2.3 Data Assumptions

The data model, as stated above, is a property graph and a directed multigraph with key/value properties on both nodes and edges [7, 1]. The building blocks in this are the following:

- Node: an entity (e.g., a person). Classified by zero or more labels (e.g., :User).
- Relationship: a connection that must have a start node, an end node, a direction, and exactly one type (e.g., FOLLOWS).
- Property: key-value pairs stored on nodes and relationships.

The relationships in this are first class relationships and stored data, not rebuilt with joins at each query time [4]. The flexible schema is not restricted to a pre-defined model and the design can change over time. The index free adjacency is each node directly references its adjacent, neighboring, nodes. Therefore, a traversal is a pointer lookup with complexity $O(1)$ compared to an index lookup with complexity $O(\log n)$ in non-native systems [8]. The paper compares this to CODASYL-style pointers, which is good for the traversal of long edge chains compared to an RDBMS which must do it via joins [1].

2.4 Natural Applications Supporting

A general rule of thumb is the answer is found between the records and in the connections rather than inside of them. Graph databases are useful for the following cases:

- Questions about relationships between elements such as navigating hierarchies, discovering hidden connections, and network analysis (finance, retail, government) [7].
- Social networks, which includes “likes”, “friends-of” and knowledge graphs [1].
- Long chains of relationships, such with many “hops”, and such as friends of friends. The cost of a query only depends on the data it touches and not the total size of the data [8].

2.5 Struggles

The biggest struggles are apps with any long-chain relationship queries. The paper argues that the value of Neo4j depends on whether there are enough "long chain" scenarios in order to justify forgoing a relational database [1].

A relational database is also an option that is always on the table. A graph can be stored as just nodes and edges. Furthermore, studies show that relational databases such as SQL/PGQ in DuckDB were up to ten times faster than a leading graph database [1]. Large scale graph analytics were also a struggle and the paper specifically mentions that this is a special case that should be done in main memory with specialized data structures [1].

Dense nodes, also known as "supernodes" are nodes with many relations, which can make data retrieval expensive. The solution to this problem is to remodel the data using intermediate nodes [9]. Horizontal scaling is another problem. Splitting a connected graph across several different machines is difficult. In addition, Neo4j struggled with scalability and added property sharding in 2025 (in Infinigraph, its self-managed offering) [10].

3 Query Language and Interface

3.1 Query and Programming Languages

One of the languages it supports is Cypher, which is a declarative graph query language. It was created in 2011 by Neo4j engineers as an SQL-equivalent for graph databases [11]. Neo4j also launched the openCypher initiative in October 2015 [12].

GQL (ISO/IEC 39075) is an ISO standard query language for property graphs and it was published on April 11, 2024 [12, 13]. It is the first ISO database query language in more than 35 years, while SQL was first published in 1987. GQL incorporates many features of openCypher, GSQL, PGQL, and G-Core, and shares its Cypher-derived pattern-matching sublanguage with SQL/PGQ [13]. Cypher also supports most mandatory GQL features and a substantial portion of the optional ones, but a few mandatory features are noticeably missing, so it is not fully conformant [14].

The official programming languages it has drivers for are Java, Python, JavaScript, .NET, and Go, which all connect over Neo4j's Bolt protocol [15].

3.2 SQL Support

Neo4j does not support SQL natively, as its native language is Cypher. However, Cypher was designed to feel like and behave like SQL, so it feels similar, such as the aggregations [11]. SQL-to-Cypher translation is also an optional feature of the Neo4j JDBC driver (`enableSQL-Translation=true`). There are limitations to this such as outer joins and mapping tables, and there is no single way to map table names to graph objects [16].

A BI Connector also lets SQL/JDBC tools, such as Tableau, read Neo4j by exposing a virtual table for each node label and each relationship. It is read only [17].

3.3 Key Differences

The main differences between Cypher and SQL are the following [11].

- Cypher describes the shape to find, (passenger:Passenger)-[:BOOKED]->(trip:Trip), instead of joining tables on keys.
- Clause order is different. SQL starts with SELECT and Cypher ends with the RETURN clause: MATCH, WHERE, RETURN.
- Conciseness is a big one. Cypher queries are oftentimes more concise than SQL.

- Variable-length paths are where Cypher succeeds. The query in Cypher is (*1..10) where in SQL it is recursive.
- The paper's view is plain "vanilla" SQL is not expressive enough for graph queries, but SQL:2023 added SQL/PGQ, which helps further narrow functionality differences between relational and native graph databases [1].

3.4 Worked Examples: el_paso Airline Database (MySQL) vs Neo4j (Cypher)

The SQL below comes from an earlier coursework exercise on this schema and was run against the live el_paso_airport database (MySQL 8.4); the Cypher gives the equivalent formulation in Neo4j.

The actual schema of el_paso_airport, verified with DESCRIBE, and the dataset size are given in Table 3.

Table 3: The el_paso_airport schema and dataset size, verified with DESCRIBE (MySQL 8.4).

Table	Key Columns	Rows
aircrafts	AircraftID, Name, NumSeats, ManufacturedDate, LastServiceDate	20
booking	PassengerID + TripID (composite PK), BookDate, BookTime	500
departments	DeptID, Name, Category	7
flies	PilotID + TripID (composite PK)	200
passengers	PassID, FirstName, LastName, Birthdate, Address, Email, PhoneNumber, RewardPoints, Citizenship, Gender	50
pilots	PilotID, FirstName, LastName, Birthdate, ..., Salary, HiredDate, Rating	20
services	ServiceID, StaffID, Staff2ID, AirCraftID, ServiceDate, Description	100
staff	StaffID, FirstName, LastName, ..., Salary, DepartmentID, HiredDate, PositionName	20
trips	TripID, AirCraftID, Capacity, DeptTown, DeptDate, DeptTime, ArrTown, ArrDate, ArrTime, Fare	200
works	StaffID + DepartmentID (composite PK), Year	20

As the row counts in Table 3 show, this dataset is small, with at most 500 rows in any table, so it illustrates how each language expresses a question rather than how the two systems perform.

Listing 1 gives the graph model for the same data, with one node per entity and one relationship per join table.

Listing 1: Property graph model for the el_paso_airport data

```
(:Passenger {passID, firstName, lastName, birthdate, email, phoneNumber, rewardPoints,
    citizenship, gender})
(:Trip {tripID, capacity, deptTown, deptDate, deptTime, arrTown, arrDate, arrTime, fare})
(:Pilot {pilotID, firstName, lastName, birthdate, phoneNumber, salary, hiredDate, rating})
(:Staff {staffID, firstName, lastName, birthdate, phoneNumber, salary, hiredDate,
    positionName})
(:Department {deptID, name, category})
(:Aircraft {aircraftID, name, numSeats, manufacturedDate, lastServiceDate})
(:Service {serviceID, serviceDate, description})

(:Passenger)-[:BOOKED {bookDate, bookTime}]->(:Trip)   // replaces the booking table
(:Pilot)-[:FLIES]->(:Trip)                             // replaces the flies table
(:Staff)-[:WORKS_IN {year}]->(:Department)             // replaces the works table
(:Trip)-[:USES]->(:Aircraft)                           // replaces trips.AirCraftID
(:Service)-[:PERFORMED_ON]->(:Aircraft)                // replaces services.AirCraftID
(:Staff)-[:PERFORMED]->(:Service)                      // replaces services.StaffID and
    Staff2ID
```

Modeling points worth making in the report:

- A composite primary key (booking, flies, works) exists only to express a many-to-many link. In the graph it disappears: the link is the relationship.
- services.StaffID and services.Staff2ID are two columns for "the staff who did this service". In a graph this is one relationship type used twice, so adding a third technician needs no schema change.
- Key point: every join table (booking, flies, works) becomes a relationship, and its extra columns (e.g. BookDate) become relationship properties.

A. Basic CRUD

Table 4 lists the four basic operations, together with insert-or-update and record linking, in MySQL and in Cypher.

Table 4: Basic CRUD operations in MySQL and the equivalent Cypher statements.

Operation	MySQL	Cypher
Create	`INSERT INTO passengers (PassID, FirstName, LastName, RewardPoints) VALUES (1, 'James', 'Bond', 2500);`	`CREATE (:Passenger {passID:1, firstName:'James', lastName:'Bond', rewardPoints:2500});`
Read	`SELECT FirstName, LastName FROM passengers WHERE RewardPoints > 2000;`	`MATCH (passenger:Passenger) WHERE passenger.rewardPoints > 2000 RETURN passenger.firstName, passenger.lastName;`
Update	`UPDATE passengers SET RewardPoints = 3000 WHERE PassID = 1;`	`MATCH (passenger:Passenger {passID:1}) SET passenger.rewardPoints = 3000;`
Delete	`DELETE FROM passengers WHERE PassID = 1;`	`MATCH (passenger:Passenger {passID:1}) DETACH DELETE passenger;`
Insert-or-update	`INSERT ... ON DUPLICATE KEY UPDATE ...`	`MERGE (passenger:Passenger {passID:1}) SET passenger.rewardPoints = 3000;`
Link 2 records	`INSERT INTO booking (PassengerID, TripID, BookDate) VALUES (1, 55, '2019-07-04');`	`MATCH (passenger:Passenger {passID:1}), (trip:Trip {tripID:55}) CREATE (passenger)-[:BOOKED {bookDate: date('2019-07-04')}]->(trip);`

- Three points follow from Table 4. DETACH DELETE deletes a node and its relationships in one step. MySQL needs ON DELETE CASCADE or several statements.
- Creating a link is CREATE (passenger)-[:BOOKED]->(trip); there is no foreign key and no join table.
- MERGE is "match or create", which has no single-statement SQL equivalent that is portable.

B. Simple Query: staff whose position is "treasurer"

Listing 2: Selecting staff by position in SQL

```sql
select FirstName, LastName, StaffID from staff where PositionName = 'treasurer';
```

Listing 3: Selecting staff by position in Cypher

```
MATCH (staff:Staff {positionName: 'treasurer'})
RETURN staff.firstName, staff.lastName, staff.staffID;
```

Verdict (Listings 2 and 3): near-identical. No graph advantage when the answer is inside one table. MySQL result (3 rows): Kara Leon (193514918), Gabriella Hardy (473287624), Gloria Brown (648060877).

C. One Join: trips booked by "James Bond" in July 2019

Listing 4: Trips booked by one passenger in July 2019, in SQL

```
select distinct booking.TripID
from booking
where booking.BookDate >= '2019-07-01' and booking.BookDate < '2019-08-01'
and booking.PassengerID in (select passengers.PassID from passengers
where passengers.FirstName = 'James' and passengers.LastName = 'Bond');
```

Listing 5: The same query in Cypher, filtering in a separate WHERE clause

```
MATCH (passenger:Passenger {firstName:'James', lastName:'Bond'})-[booking:BOOKED]->(trip:Trip
    )
WHERE booking.bookDate >= date('2019-07-01') AND booking.bookDate < date('2019-08-01')
RETURN DISTINCT trip.tripID;
```

Verdict (Listings 4 and 5): the sub-query disappears. The filter on bookDate sits on the relationship.

Three ways to filter in Cypher:

1. Property map inside the pattern: equality only. A map "can be added to filter on properties of the relationship based on equality" [18].

Listing 6: Filtering with a property map inside the pattern

```
MATCH (passenger:Passenger {firstName:'James', lastName:'Bond'}) - [booking:BOOKED {bookDate:
     date('2019-07-04')}]->(trip:Trip)
```

2. Inline WHERE inside the brackets — any predicate, on nodes (MATCH (station:Station WHERE station.name STARTS WITH 'Preston')) and on relationships (-[r WHERE time() + duration(r.duration) < time('22:00')]->) [18].

Listing 7: Filtering with an inline WHERE inside the relationship brackets

```
MATCH (passenger:Passenger {firstName:'James', lastName:'Bond'})
-[booking:BOOKED WHERE booking.bookDate >= date('2019-07-01') AND booking.bookDate < date('
    2019-08-01')]->
(trip:Trip)
RETURN DISTINCT trip.tripID;
```

3. Separate WHERE clause: the familiar SQL-like form, used in the query above.

In SQL every filter goes in one WHERE clause, far from the join that produced the row. Cypher lets the filter sit on the element it constrains, inside the pattern. Relationship pattern predicates are a relatively recent addition to Cypher.

D. Aggregation: pilots who flew exactly once in July 2019

Listing 8: Pilots who flew exactly once in July 2019, in SQL

```
select pilots.FirstName, pilots.LastName, pilots.PhoneNumber
from pilots
where pilots.PilotID in (select flies.PilotID from flies
where flies.TripID in (select trips.TripID from trips
where trips.DeptDate >= '2019-07-01' and trips.DeptDate < '2019-08-01')
group by flies.PilotID having count(flies.TripID) = 1);
```

Listing 9: The same query in Cypher, using WITH in place of GROUP BY and HAVING

```
MATCH (pilot:Pilot)-[:FLIES]->(trip:Trip)
WHERE trip.deptDate >= date('2019-07-01') AND trip.deptDate < date('2019-08-01')
WITH pilot, count(DISTINCT trip) AS tripCount
WHERE tripCount = 1
RETURN pilot.firstName, pilot.lastName, pilot.phoneNumber;
```

Verdict (Listings 8 and 9): WITH is Cypher's pipeline step; it does the job of GROUP BY ... HAVING. Note that Cypher groups implicitly by the non-aggregated variables, so there is no GROUP BY clause.

E. Two Hops: passengers to El Paso with more than 2000 reward points

Listing 10: Passengers to El Paso with more than 2000 reward points, in SQL

```
select passengers.FirstName, passengers.LastName, passengers.RewardPoints
from passengers
where passengers.RewardPoints > 2000
and passengers.PassID in (select booking.PassengerID from booking
where booking.TripID in (select trips.TripID from trips
where trips.ArrTown = 'El Paso'))
order by passengers.RewardPoints;
```

Listing 11: The same query in Cypher, two nested sub-queries collapsed into one pattern

```
MATCH (passenger:Passenger)-[:BOOKED]->(:Trip {arrTown:'El Paso'})
WHERE passenger.rewardPoints > 2000
RETURN DISTINCT passenger.firstName, passenger.lastName, passenger.rewardPoints
ORDER BY passenger.rewardPoints;
```

Verdict (Listings 10 and 11): two nested sub-queries collapse into one pattern. MySQL result: 12 passengers, from Raul Dee (2178 points) to Jose Gusler (4879).

F. Deep Chain: services on aircraft flown by pilot "Parth Nagarkar"

Listing 12: Services on aircraft flown by one pilot, in SQL: four levels of nested sub-queries

```
select count(distinct services.ServiceID)
from services
where services.AirCraftID in (select trips.AirCraftID from trips
where trips.TripID in (select flies.TripID from flies
where flies.PilotID in (select pilots.PilotID from pilots
where pilots.FirstName = 'Parth' and pilots.LastName = 'Nagarkar')));
```

Listing 13: The same question in Cypher, expressed as a single path

```
MATCH (:Pilot {firstName:'Parth', lastName:'Nagarkar'})-[:FLIES]->(:Trip)-[:USES]->(aircraft:
    Aircraft)<-[:PERFORMED_ON]-(service:Service)
RETURN count(DISTINCT service.serviceID);
```

Verdict (Listings 12 and 13): this is the clearest example of the difference. Four levels of nested sub-queries become one line that reads like the path itself: pilot → trip → aircraft → service. The MySQL result is 28.

G. Natural Join: pilots on high-capacity trips departing from a town starting with "S"

Listing 14: High-capacity trips from towns beginning with S, in SQL, using NATURAL JOIN

```
select LastName, TripID, ArrTown
from trips natural join flies natural join pilots
where Capacity > 450 and DeptTown like 'S%';
```

Listing 15: The same query in Cypher, naming the relationship explicitly

```
MATCH (pilot:Pilot)-[:FLIES]->(trip:Trip)
WHERE trip.capacity > 450 AND trip.deptTown STARTS WITH 'S'
RETURN pilot.lastName, trip.tripID, trip.arrTown;
```

MySQL result (6 rows): Wollenberg, Hess, Ledbetter, Murphy, Hess, Schmidt. Verdict (Listings 14 and 15): NATURAL JOIN silently joins on every column that shares a name, which is risky. Cypher names the relationship explicitly, so the link is never guessed. Also note STARTS WITH in place of LIKE 'S%'.

H. What SQL Cannot Express as Easily: variable-length paths

There is no equivalent example above, because the schema has no recursive relationship. Example question: "which trips are reachable from this one by transfers, up to 5 legs?"

Listing 16: A variable-length path in Cypher, which SQL expresses with a recursive CTE

```
MATCH path = (startTrip:Trip {tripID:55})-[:CONNECTS*1..5]->(destinationTrip:Trip)
RETURN destinationTrip.tripID, length(path);
```

In MySQL the query in Listing 16 needs a recursive CTE (WITH RECURSIVE), and the query grows much more complex as the depth increases. This is the clearest case in which the graph model expresses a question that SQL can only reach indirectly.

4 Use Cases and Practical Application

4.1 Business Use Cases

Real-time recommendations are key to the success of any online business and its ability to instantly capture any new interests shown by customer behavior, something batch processing can't accomplish compared to Neo4j.

Benefits

The benefits of using Neo4j are:

- **Personalized user recommendations.** Graphs make it possible to recommend things to users in real time, using both explicit connections (like followed accounts or friendships) and links between facts that look unrelated at first, which can reveal likely interests. By combining a user's profile, stated preferences, and past behavior, such as purchases, a system can suggest relevant products, services, content, or even other people.

- **Multi-criteria search.** Users can search for products, services, or people using many detailed filters at once. Because the graph model is flexible, new data sources and data types can be added over time to improve the results, without having to heavily redesign the underlying data model.

4.2 Industry Use Cases

The telecommunications market is all about the customer. So, graph databases are an excellent fit for modeling, storing, and querying telecommunications data and managing complex network structures, in addition to gaining customer satisfaction and competitive advantage.

Benefits

The benefits of using Neo4j are:

- **Impact analysis and network planning.** Neo4j helped them to quickly determine the impact of node failure, maintenance outages, and rerouting to other components.
- **Root cause analysis.** Immediately identify the root cause of the problem by tracking back dependencies quickly and easily.
- **Routing and quality of service (QoS) mapping.** Neo4j helped them to navigate the best, shortest, or least busy path and pinpoint the best location to introduce a new service.

4.3 Healthcare and Life Science

In the modern biology world, genes, proteins, cells, and tissues are naturally connected. As a result, companies like Novartis and ChemAxon use Neo4j's technology to analyze these complicated relationships.

Benefits

The benefits of using Neo4j are:

- **Drug discovery.** Linking biological and chemical data speeds up R&D (Novartis, ChemAxon).
- **Patient journey mapping.** Tracks disease progression and helps prevent poor outcomes.
- **Genetic tracking.** Monsanto tracked corn lineages to breed better crops.

4.4 Real Use Case: WestJet

Problem

WestJet's problem was managing a complex network of flights, schedules, routes, seasons, and airport connections. Customers needed accurate information about available travel options, but determining valid routes required analyzing many interconnected relationships. Relational databases required numerous table joins to process these relationships, making the system difficult to manage as the airline expanded.

Solution

Neo4j was chosen because its graph structure efficiently models and traverses relationships among airports, flights, schedules, and destinations, enabling faster and more flexible route searches.

To address the challenges of managing complex flight schedules, WestJet implemented a Neo4j graph database solution and developed an API that could automatically determine valid flight routes and itinerary combinations. The system was designed to process IATA Standard Schedules Information Manual (SSIM) data, reducing the manual effort required to update schedule information and communicate flight availability across the organization.

The solution incorporated important business rules, including minimum connection times for domestic and international flights, customs requirements, seasonal route availability, and day-of-week scheduling restrictions. By representing airports, flights, and schedules as interconnected nodes and relationships, Neo4j enabled efficient graph traversal to identify valid routes and eliminate invalid travel combinations.

WestJet initially built the solution using Neo4j Community Edition and later upgraded to Neo4j Enterprise Edition to improve performance. The upgrade significantly reduced processing time for schedule updates and route calculations. In 2024, the airline migrated to Neo4j AuraDB on Microsoft Azure, gaining improved scalability, reliability, security, and reduced database maintenance. Overall, the graph-based solution streamlined schedule management, accelerated route analysis, and provided accurate flight information for both employees and customers.

4.5 Companies Using Neo4j

Yes, according to Neo4j, there are 1700+ organizations that rely on Neo4j, and 84 of the Fortune 100 companies are Neo4j customers. The notable companies are:

- **WestJet** — a flight company that uses Neo4j for the booking experience.
- **Cisco** and **Comcast** — telecom companies.
- **Transport for London** — reported a 10% reduction in congestion with a Neo4j-based digital twin.
- **BNP Paribas Personal Finance** — reported a 20% reduction in fraud.

4.6 How and Why Does Neo4j Work Better for Their Use Cases Compared to Traditional Relational Databases?

Neo4j stores relationships directly alongside the data (nodes and relationships), so a query follows links from one node to the next instead of computing joins between tables at query time.

- **Faster query speed.** Neo4j is particularly well suited to queries involving complex, multi-hop relationships. Since relationships are explicitly represented and traversed in the graph, these queries can be more natural and efficient than equivalent queries that require multiple joins in a relational database. However, relational databases can be more appropriate for highly structured tabular data, aggregation-heavy workloads, and traditional transactional applications.
- **Understand deeper relationships.** Graphs make it easier and more accessible to understand deeper relationships, and an important advantage is that siloed datasets can be brought together to support deeper insights in a company. In traditional databases, integrating siloed and distributed datasets is more complex and challenging to do.
- **Schema flexibility.** In a graph database, changing the schema is easier and has minimal impact on query performance. Changing business needs leads to frequent schema changes and application or database redesign.
- **Intuitive query language.** Cypher queries are easy to understand in a graph database and lead to reduced chances of errors. In traditional databases, SQL queries are complex for many recursive traversals and joins and lead to confusion and errors.

5 Competitors and Market Analysis

5.1 ArcadeDB

ArcadeDB is an open-source multi-model database that combines graph, document, key/value, search, time-series, and vector data in one engine.

- **Query languages:** SQL, Cypher, Gremlin, GraphQL, and MongoDB Query Language.
- **License:** Apache 2.0, free for commercial use.
- **AI integration:** built-in MCP server for LLM and AI agent integration.
- **Graph analytics:** 70+ built-in graph algorithms.
- **Performance:** the vendor claims high performance, but independent benchmarks are limited.
- **Compared with Neo4j:** it is free and multi-model, but Neo4j has a larger community and ecosystem.

5.2 Memgraph

- **Query language:** Cypher, and it is described as Neo4j-compatible.
- **License:** Community Edition under the BSL (source-available), Enterprise Edition under a commercial license.
- **Streaming:** built-in ingestion from Kafka, Pulsar, SQL, and CSV.
- **Performance:** the vendor claims sub-millisecond traversals, but independent benchmarks are limited.
- **Compared with Neo4j:** it has the same query language and a faster in-memory design for real-time workloads. However, graph size is limited by available RAM (a third-party review lists this as a drawback), and its ecosystem is smaller than Neo4j's.

5.3 How Does Neo4j Differ?

Neo4j differs from many of its competitors because it is a native graph database designed specifically for storing and analyzing highly connected data. Unlike traditional relational databases such as MySQL, which rely on tables and joins, Neo4j represents data as nodes and relationships, allowing it to efficiently traverse complex connections. Neo4j also uses the Cypher query language, which is designed to make graph queries more intuitive and easier to write. Additionally, features such as index-free adjacency enable faster relationship traversal, making Neo4j particularly effective for applications such as recommendation systems, fraud detection, social networks, route optimization, and knowledge graphs. While other databases may support graph-like queries, Neo4j is built specifically for graph workloads and relationship-centric analysis.

5.4 What Makes Neo4j Unique?

Neo4j is unique because it is a native graph database that stores data as nodes and relationships rather than tables and rows. One of its most distinctive features is **index-free adjacency**, which allows nodes to directly reference connected nodes, enabling efficient graph traversal without requiring complex joins. Neo4j also provides the **Cypher query language**, which is specifically designed for expressing and querying relationships in a simple and intuitive way. These features make Neo4j particularly effective for analyzing highly connected data, such

as social networks, recommendation systems, fraud detection, route planning, and knowledge graphs, where traditional relational databases can become complex and less efficient.

6 Drawbacks and Limitations

6.1 Memory Bound on a Single Machine

Independent research shows that standalone Neo4j is limited by the physical memory of its host server. In a comparison with Apache Spark, Neo4j ran PageRank faster on small datasets but failed on the largest graph because of a RAM overflow, whereas Spark's distributed setup completed every run [19].

Neo4j's speed on connected data comes with a trade-off: it consumes more memory than relational databases such as MySQL because it relies on in-memory traversal and caching, and SQL still scales better for aggregation-heavy workloads.

6.2 Hard-To-Predict Query Performance

Neo4j has no standard query benchmark, and SQL-based prediction methods cannot be applied directly to Cypher. A 2022 study found that even a purpose-built neural model predicted Cypher execution time with 12 to 16% average error, which makes workload planning and scheduling harder [20].

Neo4j is not recommended when data is primarily tabular and relationships are not a major part of the application. In systems such as payroll, inventory management, customer records, or student databases, relational databases are often more efficient and easier to manage.

It is also less suitable for applications that mainly perform simple CRUD (Create, Read, Update, Delete) operations or require large-scale reporting and aggregation, such as sales dashboards, financial reports, and business intelligence systems. In these cases, relational and columnar databases generally provide better performance.

Neo4j is not the best choice for full-text search, document repositories, or storing large files such as images, videos, and PDFs. Specialized search engines and storage systems are better suited for these tasks. Similarly, high-volume data streams such as IoT sensor data, application logs, and clickstream data often benefit more from time-series or distributed databases.

In addition, Neo4j can be challenging to scale horizontally because graph relationships are difficult to partition across multiple servers. It may also be less effective when queries involve only shallow, fixed-depth relationships that can be handled efficiently using traditional SQL joins.

Overall, Neo4j is most valuable for highly connected data and complex relationship analysis. When relationships are not the primary focus, relational, analytical, or specialized databases are usually a better option [21].

7 Phase 2 Readiness and Reflection

Based on our analysis, Neo4j is a strong candidate to compare against MySQL because both databases can store and query connected data, but they handle relationships differently. Neo4j is specifically designed for relationship-heavy data and can efficiently traverse complex connections using its graph structure. In contrast, MySQL stores data in tables and often requires multiple joins to analyze relationships, which can become complex and computationally expensive as the number of connections grows.

At the same time, MySQL is highly optimized for tabular data, transactional workloads, and aggregation queries, making it a strong benchmark for comparison. Comparing Neo4j and MySQL allows us to evaluate how graph databases perform relative to traditional relational

databases when handling complex relationship queries, route analysis, and connected datasets. However, the comparison may also reveal challenges, as Neo4j and MySQL are designed for different types of workloads and may excel in different scenarios.

It is expected for Neo4j to handle complex relationship queries more efficiently than MySQL. MySQL can perform the same tasks, but it often requires multiple joins, which may increase query complexity and execution time as the dataset grows.

Furthermore, Neo4j can execute relationship-heavy queries with fewer query statements than MySQL. As the number of joins increases, MySQL queries may become more complex and take longer to execute. We also expect Cypher queries in Neo4j to be easier to write for connected data.

8 Group Members and Contributions

- **Rigesh Krishnaraj**
  - Researched and wrote *Use Cases and Practical Applications*: the primary use cases of Neo4j, companies using it, and why it works better for them than traditional relational databases.
  - Researched and wrote *Competitors and Market Analysis*: the literature survey of academic and industry sources, how Neo4j differs from its competitors, and what is novel about it.
  - Researched and wrote *Drawbacks and Limitations*: the major limitations of Neo4j and the scenarios in which it would not be recommended.
  - Researched and wrote the *Phase 2 Readiness Reflection*: why Neo4j is a suitable candidate to compare against MySQL and the differences expected experimentally.
- **Suresh Hemal Warnasooriya Warnakulasooriya Mudiyanselage**
  - Researched and wrote the *Introduction*: who created Neo4j, when it was created, and why.
  - Researched and wrote *Data Model Fit*: the data model assumptions Neo4j makes, the application types the model supports well, and those it struggles with.
  - Researched and wrote *Query Language and Interface*: the query and programming languages Neo4j supports, its level of SQL support, and the key differences between Cypher and SQL, including the worked MySQL versus Cypher examples on the el_paso_airport database.
- **Scott C. Robertson**
  - Analyzed the research submitted by all members and compiled it into the final report.
  - Edited the report for structure, consistency and clarity, and prepared the final document for submission.
  - Verified the accuracy of the material submitted by the other members and made adjustments where it was incorrect or unclear.
  - Produced the final publishable version of the report.

References

[1] M. Stonebraker and A. Pavlo, "What goes around comes around... and around...," *SIGMOD Record*, vol. 53, no. 2, pp. 21–37, Jun. 2024, https://db.cs.cmu.edu/papers/2024/whatgoesaround-sigmodrec2024.pdf.

[2] Wikipedia, "Neo4j," Wikipedia, The Free Encyclopedia, https://en.wikipedia.org/wiki/Neo4j.

[3] E. Eifrem, "Hello, world: Neo4j, Inc." Neo4j Blog, Jul. 2017, https://neo4j.com/blog/emil-eifrem/hello-world-neo4j-inc/.

[4] Carnegie Mellon University, "Neo4j," Database of Databases, https://dbdb.io/db/neo4j.

[5] D. Baer, ""something much broader than compensation drives us." — an interview with Emil Eifrem, Neo4j founder," HackerNoon, Jun. 2019, https://hackernoon.com/something-much-broader-than-compensation-drives-us-emil-eifrem-neo4j-founder-interview-168278957077.

[6] Ø. Startups, "Ola Rollén invests in Malmö-founded Neo Technology," Øresund Startups, Nov. 2016, https://oresundstartups.com/ola-rollen-invests-malmo-founded-neo-technology/.

[7] Neo4j, "What is a graph database," Neo4j Getting Started Guide, https://neo4j.com/docs/getting-started/graph-database/.

[8] J. Stegeman, "Native vs. non-native graph database," Neo4j Blog, May 2023, https://neo4j.com/blog/cypher-and-gql/native-vs-non-native-graph-technology/.

[9] Neo4j, "Modeling designs," Neo4j Getting Started Guide, https://neo4j.com/docs/getting-started/data-modeling/modeling-designs/.

[10] L. Clark, "Neo4j intros 'property sharding' to tackle scalability," The Register, Sep. 2025, https://www.theregister.com/2025/09/11/neo4j_property_sharding_to_address_scalability_challenge/.

[11] Neo4j, "Cypher overview," Neo4j Cypher Manual, https://neo4j.com/docs/cypher-manual/current/introduction/cypher-overview/.

[12] V. Malenchino, "openCypher will pave the road to GQL for Cypher implementers," Neo4j Blog, May 2024, https://neo4j.com/blog/cypher-and-gql/opencypher-gql-cypher-implementation/.

[13] K. W. Hare, "ISO/IEC 39075 database language GQL," ISO/IEC JTC 1, Apr. 2024, https://www.jtc1info.org/wp-content/uploads/2024/04/2024-Article-39075-Database-Language-GQL.docx.pdf.

[14] Neo4j, "GQL conformance," Neo4j Cypher Manual, https://neo4j.com/docs/cypher-manual/current/appendix/gql-conformance/.

[15] ——, "Neo4j drivers," Bolt Protocol Documentation, https://neo4j.com/docs/bolt/current/neo4j-drivers/.

[16] ——, "SQL to Cypher translation," Neo4j JDBC Driver Manual, https://neo4j.com/docs/jdbc-manual/current/sql2cypher/.

[17] D. Allen and L. Smith, "Getting started with the Neo4j BI Connector," Neo4j Blog, Mar. 2020, https://neo4j.com/blog/cypher-and-gql/getting-started-with-the-neo4j-bi-connector/.

[18] Neo4j, "Fixed-length patterns," Neo4j Cypher Manual, https://neo4j.com/docs/cypher-manual/current/patterns/fixed-length-patterns/.

[19] I. Ballas, V. Tsakanikas, E. Pefanis, and V. Tampakas, "Assessing the computational limits of GraphDBs' engines — a comparison study between Neo4j and Apache Spark," in *Proceedings of the 24th Pan-Hellenic Conference on Informatics*, Nov. 2020, pp. 428–433.

[20] Z. He, J. Yu, and B. Guo, "Execution time prediction for Cypher queries in the Neo4j database using a learning approach," *Symmetry*, vol. 14, no. 1, p. 55, 2022.

[21] A. Mussin, "Performance comparison of relational and graph databases," Master's thesis, Česká zemědělská univerzita v Praze, Provozně ekonomická fakulta, 2025.